

BayLore: Exploring a Geo-Historical Archive

Siddharth Bawankule

sb68@illinois.edu

University of Illinois Urbana-Champaign

Champaign, Illinois, USA

Abstract

BayLore is an interactive geo-historical exploration platform designed to make Bay Area history more accessible through map-based visualization and conversational AI. The system combines an interactive Leaflet-powered geographic interface with curated historical landmark data and an AI chatbot to help users explore culturally and historically significant locations in the San Francisco Bay Area.

Built using React, TypeScript, Tailwind CSS, Supabase, and OpenAI integration, BayLore serves as an educational tool for students, residents, historical hobbyists, and tourists seeking contextual historical insights. This paper describes the motivation, design, implementation, deployment, and evaluation of the BayLore project.

1 Introduction

The San Francisco Bay Area contains a vast network of historically significant landmarks, cultural institutions, and geographically meaningful sites that reflect centuries of social, political, and technological change. However, discovering and understanding these locations often requires navigating fragmented sources such as tourism websites, historical archives, and search engines that rarely provide integrated geographic and contextual exploration.

BayLore is an interactive geo-historical platform designed to make Bay Area history more accessible through a unified digital experience. By combining an interactive map interface with curated historical landmark data and conversational AI, BayLore allows users to explore historically relevant sites spatially while also engaging with contextual historical narratives.

The platform transforms local historical discovery from static information retrieval into an exploratory educational process. Through map-based navigation, landmark visualization, and AI-powered question answering, BayLore provides users with a more intuitive and engaging way to interact with regional history.

2 Motivation and Intended Users

BayLore was motivated by the need for a more engaging and accessible approach to local historical education. Traditional historical resources often prioritize static text or isolated documentation, which can make regional history difficult to explore geographically. For many users, understanding the historical significance of a place is enhanced when information is connected directly to physical location and interactive inquiry.

The project was specifically designed to combine digital humanities principles with modern web technologies in order to improve user engagement. Geographic visualization encourages exploration, while conversational AI supports personalized learning by allowing users to ask follow-up questions about landmarks, events, or historical context.

BayLore is intended to serve multiple audiences, including:

- Students seeking accessible educational tools
- Tourists exploring Bay Area history
- Residents interested in local cultural heritage
- History enthusiasts seeking place-based discovery
- Educators integrating digital tools into historical learning

By focusing on accessibility, educational value, and technological interactivity, BayLore aims to demonstrate how software systems can enhance public engagement with regional history.

3 System Features

BayLore provides a set of features designed to support interactive historical exploration through geographic visualization and AI-assisted learning. The system emphasizes accessibility, educational engagement, and intuitive user experience by combining multiple functionalities into a unified platform.

3.1 Interactive Geo-Historical Map

At the core of BayLore is an interactive map interface built to visually organize historically significant Bay Area landmarks by geographic location. Users can navigate the map spatially, zoom into specific regions, and discover nearby historical sites through location-based interaction. This design transforms historical exploration into an intuitive geographic experience rather than requiring users to search through disconnected resources manually.

3.2 Landmark Markers and Historical Popups

BayLore uses landmark markers to represent historically significant sites across the Bay Area. Each marker acts as a clickable interface element that provides access to curated historical information, allowing users to quickly explore site relevance, cultural significance, and contextual details. Interactive modal panels present landmark descriptions, historical narratives, and supplementary visuals, creating a streamlined educational workflow that encourages exploration.

3.3 AI-Powered Historical Chatbot

To complement map-based discovery, BayLore includes an AI chatbot that enables users to ask questions about Bay Area history, landmarks, and broader cultural context. This feature supports personalized inquiry by allowing users to move beyond static landmark summaries and engage in deeper historical discussion. By integrating conversational AI, the platform supports a more dynamic and user-driven learning experience.

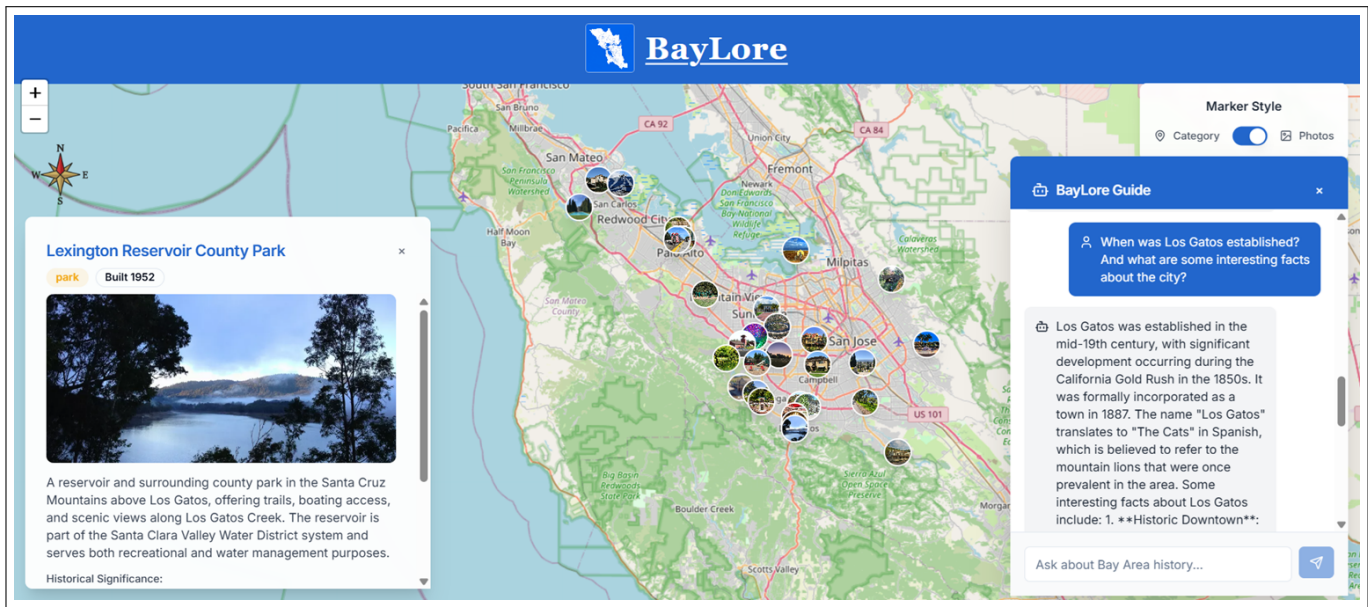


Figure 1: BayLore full interface showing map-based landmark exploration, dynamic landmark details, and the AI-powered BayLore Guide.

3.4 Visual Customization and User Interface Design

BayLore includes interface customization features such as toggling between symbolic icons and landmark-specific imagery to support different user preferences. The user interface was designed with accessibility and usability in mind, using a modern web stack to create responsive interactions, clean visual presentation, and educational clarity.

3.5 Educational and Exploratory Integration

Rather than functioning solely as a map or chatbot, BayLore integrates geographic exploration with historical education. Users can discover locations spatially, learn through curated site information, and deepen understanding through AI-assisted questioning. This multi-layered system architecture distinguishes BayLore from traditional static historical websites by encouraging active engagement and exploratory learning.

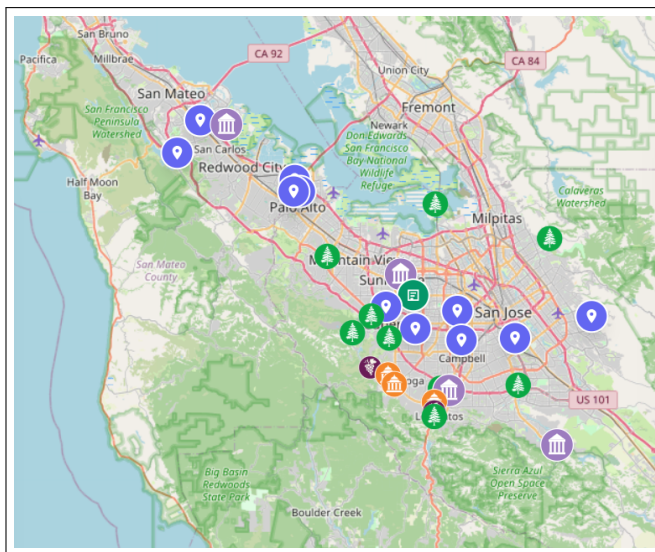


Figure 2: BayLore map displaying symbolic landmark markers categorized by type.

4 System Architecture and Implementation

BayLore is implemented as a full-stack web application that combines a modern React-based frontend, a Supabase-backed data layer, and an AI-powered conversational interface. The architecture separates user interaction, landmark data storage, and chatbot response generation so that each component can be developed and extended independently.

4.1 Frontend Architecture

The frontend is built using React, TypeScript, Vite, Tailwind CSS, and shadcn-ui. React provides the component structure for the map, landmark panels, marker controls, and chatbot interface, while TypeScript improves reliability by enforcing typed data structures for landmarks and user interactions. Vite supports fast local development and bundling, and Tailwind CSS is used to create a responsive interface with consistent styling.

The map interface is implemented using Leaflet through React Leaflet. Landmark records are rendered as map markers based on their stored latitude and longitude values. User interactions such as clicking a marker, switching marker styles, or opening the chatbot update the relevant React state and dynamically change what is displayed on the page.

4.2 Backend Database Infrastructure

BayLore stores landmark information in a Supabase-managed PostgreSQL database through a structured landmarks table that organizes geographic, historical, and media-related information for each location. Core fields such as title, latitude, longitude, and summary support map rendering and landmark discovery, while category labels organize landmarks by type for visual grouping and interface logic.

Beyond these primary fields, the full schema also includes extended metadata such as year built, architect, historical significance, curated fun facts, image URLs, website URLs, and system timestamps. Together, these fields allow BayLore to support both geographic interaction and richer historical documentation.

id	title	latitude	longitude	summary	category
06a826b-e214-4d4e-9c9f-d88f	Ainsley House	37.28874070	-121.94617000	A preserved 1920s historic home in Camp	Historic Site
25e8fde-0416-4238-9a85-e1e1	Alcatraz Island	37.82670000	-122.42330000	Former federal prison that housed infamo	Prison
c7a6a024-2773-46f4-8ea7-26d6	Allied Arts Guild	37.65480000	-122.16370000	A historic arts complex founded in 1938-1	Historic Site
1a28f914-d26a-4887-9043-f1f1	Almaden Lake Park	37.24079670	-121.87059620	A large neighborhood park in San Jose ca	Park
03ec18c3-63b4-46da-9727-60d4	Alum Rock Park	37.39474000	-121.83030000	A large municipal park in San Jose locate	Park
814f18c0-1d55-4933-8f21-4825a	Alviso Marina County Park	37.43042320	-121.97947920	A 20.6-acre bayside park along Alviso Sl	Park
4c17fd49-74dc-4354-a87a-518f	Apple Park	37.33449000	-122.00900000	Apple's 2.8 million square foot circular h	Tech Landmark
9b615c37-1245-440a-a80f-579c	Beauchamps Park	37.29056370	-122.03956810	A neighborhood park in Saratoga featurin	Park
42a3f162-6f8f-4b40-839f-c3a1	Coit Tower	37.80204000	-122.40580000	Built in 1933 with funds from Lillie Hithco	Monument
05c4a8d0-4b88-4a9e-b459-03	Communications Hill	37.29023530	-121.87577650	A hillside neighborhood and elevated are	Landmark
a4604f06-93ab-4523-a59e-99d1	Evergreen Village Square	37.33385580	-121.77461940	A neighborhood commercial and commu	Landmark

Figure 3: Cropped view of the BayLore landmarks table showing example records and selected visible fields from the broader database schema.

The visible fields shown in Figure 3 demonstrate how landmark records are structured for frontend rendering, while the complete schema additionally supports deeper historical metadata and external resource integration. Indexed category and location fields improve retrieval efficiency, allowing BayLore to organize landmarks effectively while maintaining scalable backend extensibility.

4.3 AI Chatbot Integration

The BayLore Guide is implemented as a context-aware conversational AI system designed specifically for Bay Area historical exploration rather than as a generic chatbot. The interface is built as a dedicated React chat component that maintains conversation history, supports real-time user interaction, and communicates with a Supabase Edge Function backend for response generation. This architecture separates frontend messaging, backend context construction, and AI inference into distinct layers, improving maintainability and modularity.

A major distinguishing feature of the AI chat is its use of dynamic app-state context injection. User queries are sent alongside current map coordinates, selected landmark data, and recent conversation history, allowing the backend to construct a specialized system prompt tailored to the user's exact exploration state. If a landmark is currently selected, the chatbot prioritizes that landmark's title, category, historical significance, and metadata when interpreting questions such as "What am I looking at?" or "Tell me more about this place." This context-sensitive design allows BayLore to function as a geo-historical guide that responds not only to general Bay Area history questions but also to specific landmarks and locations currently being explored.

The backend prompt architecture also enforces domain specialization and historical accuracy. BayLore's system prompt explicitly restricts responses to San Francisco Bay Area history, culture, geography, architecture, and heritage topics while prioritizing factual reliability and uncertainty acknowledgment over speculation. By combining landmark-specific context, geographic awareness, and conversation memory with OpenAI's GPT-4o-mini model, the BayLore Guide transforms static landmark exploration into adaptive historical inquiry. This integration enables users to move fluidly between map-based discovery and personalized educational dialogue, making the chatbot a central component of BayLore's broader digital humanities framework.

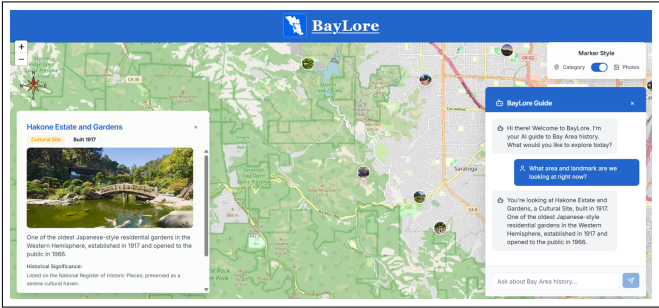


Figure 4: BayLore Guide using selected landmark context to answer user questions based on the current map state.

4.4 Internal AI-Assisted Landmark Discovery Tooling

In addition to its public-facing chatbot, BayLore includes internal AI development tooling designed to support future landmark discovery and dataset expansion. This system leverages LangGraph-based workflows to assist in identifying, organizing, and contextualizing potential historical landmarks for future inclusion in the BayLore archive.

While this tooling is not currently exposed as a direct user-facing feature, it represents an important architectural component for long-term scalability. By supporting semi-automated landmark research and content development, this internal AI subsystem helps extend BayLore beyond a static dataset toward a more expandable geo-historical platform.

This design reflects BayLore's broader goal of combining interactive public education with scalable historical knowledge infrastructure.

4.5 Historical Source Ingestion and Retrieval Infrastructure

In addition to BayLore's primary conversational chatbot, the project includes separate backend infrastructure for ingesting and semantically searching historical source materials. This subsystem supports the extraction of text from historical documents and maps, chunking and embedding of extracted content, and vector-based retrieval through Supabase.

Although this retrieval pipeline is not yet fully integrated into BayLore's primary chatbot workflow, it establishes a technical foundation for future retrieval-augmented generation (RAG). This architecture positions BayLore for future expansion beyond curated landmark metadata by enabling source-grounded historical search across archival materials.

4.6 Deployment

BayLore is publicly deployed through Vercel at baylore.vercel.app, which hosts the most recent stable published version of the platform for public use.

The GitHub repository serves as a more experimental development environment containing ongoing updates, testing, and feature iteration. Users interested in local development or exploring the codebase can clone the repository, run `npm install` to install dependencies, and launch the development server with `npm run dev`.

GitHub Repository Link: github.com/siddharthb52/baylore.

5 Key Algorithms and Technical Workflows

This section highlights several of BayLore's most important implementation workflows through simplified pseudocode derived directly from the deployed system architecture and backend functions. These workflows illustrate how BayLore processes context-aware user interaction, ingests historical archival materials, and prepares source-grounded retrieval infrastructure for future expansion.

5.1 BayLore Guide Context-Aware Chatbot Workflow

BayLore's chatbot pipeline combines selected landmark state, geographic context, and recent conversation history before generating responses.

BayLore Guide Chat Workflow

- (1) Receive user message, selected landmark, current map location, and full conversation history
- (2) Trim user message and reject empty input
- (3) Check whether message matches current-view queries (e.g., "What am I looking at?")
- (4) If current-view query:
 - If landmark is selected, return landmark-specific answer
 - Otherwise, return no-current-landmark response
- (5) Build app context:
 - Selected landmark details (or no-landmark-selected state)
 - Current map coordinates, if available
- (6) Slice the 10 most recent conversation messages
- (7) Convert messages into valid chat roles (user or assistant)
- (8) Remove malformed or empty history entries
- (9) Append current user query if not already included
- (10) Construct BayLore system prompt with:
 - Bay Area historical domain rules
 - Current app context
 - Conversation history
- (11) Send structured prompt to OpenAI API
- (12) Return BayLore response to frontend chat interface

Figure 5: Simplified BayLore Guide workflow based on the bay-area-chat Edge Function implementation.

5.2 Landmark Discovery Agent Workflow

BayLore also includes an internal landmark discovery agent for expanding the landmark dataset. The agent accepts a target city, example landmark records, and a desired number of new landmarks. It then runs a manual LangGraph-inspired state machine that plans candidate landmarks, researches them through Wikipedia and Wikimedia, enriches their metadata, evaluates quality, and saves accepted results into a proposed landmarks table.

Landmark Discovery Agent Workflow

- (1) Receive city, example landmarks, and target count
- (2) Initialize discovery state with empty candidate, accepted, and rejected lists
- (3) Run planner node to generate candidate landmark names
- (4) While target count is unmet, candidates remain, and max iterations is not reached:
 - Select next candidate
 - Search Wikipedia for extract, coordinates, and source URL
 - Retrieve Wikimedia image when available
 - Reject candidate if no page, outside Bay Area bounds, or duplicate
 - Enrich candidate with summary, category, historical significance, and fun facts
 - Critic node returns ACCEPT or REJECT decision with reasoning
 - Router saves accepted landmarks to `landmarks_proposed`
 - Add rejected landmarks to rejected list with reason
- (5) Return verified landmarks and rejected candidates

Figure 6: Simplified workflow of BayLore's internal landmark discovery agent.

5.3 Historical Source Ingestion Pipeline

Historical Source Ingestion Workflow

- (1) Receive historical source ID
- (2) Retrieve source metadata and original file URL from Supabase
- (3) Extract full text from source image/document using GPT-4 Vision OCR
- (4) Chunk extracted text into structured searchable sections
- (5) Generate embeddings for each chunk using `text-embedding-3-small`
- (6) Attach source metadata context to each embedding
- (7) Insert chunked records into `historical_chunks` table

Figure 7: Simplified historical source ingestion workflow based on BayLore's ingest-historical-source backend function.

Together, these workflows demonstrate that BayLore's technical design extends beyond a conventional interactive map by combining frontend geographic interaction, context-aware conversational AI, historical source ingestion infrastructure, and an internal LangGraph-inspired landmark discovery agent for future dataset expansion into a broader geo-historical software ecosystem.

6 Evaluation

BayLore was evaluated primarily through iterative manual functional testing conducted throughout development. Because the project emphasizes software implementation and deployment rather than formal benchmark experimentation, evaluation focused on validating frontend behavior, backend function correctness, subsystem integration, and reproducibility across both deployed and local environments.

6.1 Frontend Functional Testing

Frontend evaluation centered on validating BayLore’s core interactive user experience. Manual testing was used to verify correct landmark marker rendering across the Bay Area map, including geographic placement accuracy, category-specific symbolic marker behavior, and optional image-preview marker modes. Marker style toggles, legend functionality, and map navigation controls were repeatedly tested to ensure that users could switch between exploration modes without disrupting interface consistency.

Landmark interaction workflows were also validated by testing marker selection, modal rendering, and historical popup accuracy across multiple landmarks. This included confirming that selected landmark information displayed correctly, that modal panels reflected the intended database content, and that user interactions remained responsive during map navigation and landmark exploration.

The BayLore Guide interface was similarly tested through repeated conversational scenarios, including landmark-specific questions, broader regional historical prompts, and current-view questions such as asking what the user is currently viewing. These tests helped verify that frontend message handling, chat rendering, and response display remained stable across varied query types.

6.2 Backend and Edge Function Validation

Backend evaluation focused on validating Supabase database integrity and Edge Function behavior through direct function input/output experimentation. Landmark database testing included verifying schema consistency, geographic coordinate integrity, category classification correctness, and successful retrieval of landmark records for frontend rendering.

The bay-area-chat Edge Function was manually tested with diverse prompt structures to confirm correct context assembly from selected landmarks, current map state, and recent conversation history. Testing also included empty-state scenarios, malformed input checks, and landmark-independent prompts to evaluate fallback behavior.

Additional backend experimentation was conducted on BayLore’s internal infrastructure systems, including historical source ingestion and the LangGraph-inspired landmark discovery agent. Historical ingestion testing validated source retrieval, OCR extraction, chunk generation, and embedding preparation workflows. Landmark discovery testing focused on candidate generation, Wikipedia retrieval, Bay Area geographic filtering, enrichment quality, and acceptance or rejection logic for proposed landmarks.

6.3 Deployment and Reproducibility

BayLore’s deployment was validated through both public and local reproducibility testing. The Vercel deployment at baylore.vercel.app was used to confirm stable public accessibility and end-user functionality. Separately, the GitHub repository was cloned into a local environment to verify reproducibility through dependency installation with `npm install` and development server execution using `npm run dev`.

This dual deployment validation process helped distinguish BayLore’s stable public release from its more experimental development repository while also confirming that the software can be independently run, explored, and extended by external users.

7 Limitations and Future Work

BayLore’s current implementation provides a strong foundation for geo-historical exploration, but two primary areas remain for continued development.

First, while the application includes a substantial curated landmark dataset, the Bay Area’s historical landscape is broad enough that many additional landmarks, local histories, and underrepresented regional narratives could still be incorporated. Continued landmark expansion would strengthen historical coverage and improve the platform’s representational depth across more communities and locations.

Second, BayLore already contains backend infrastructure for historical source ingestion, embedding generation, and semantic retrieval, but these systems currently function primarily as supporting or experimental components rather than direct contributors to the main BayLore Guide chatbot experience. As a result, most chatbot responses remain driven primarily by curated landmark context and application state rather than dynamically retrieved archival material.

Future work will therefore focus on expanding BayLore’s landmark corpus while more fully integrating its existing historical ingestion and vector search systems into the user-facing conversational workflow. Together, these improvements would deepen BayLore’s historical coverage while also allowing the platform to move toward richer source-grounded historical responses.

8 Member Contribution

This is a solo project. Siddharth Bawankule independently conceived, designed, implemented, deployed, and evaluated BayLore, including frontend development, backend integration, AI chatbot implementation, historical data curation, and system testing.

A BayLore Landmarks Table Schema

This appendix provides the full SQL schema definition for BayLore's primary landmarks table, including field definitions, constraints, and indexing strategy used to support efficient category-based and geographic retrieval.

```
create table public.landmarks (  
  id uuid not null default gen_random_uuid (),  
  title text not null,  
  latitude numeric(10, 8) not null,  
  longitude numeric(11, 8) not null,  
  summary text not null,  
  category text not null,  
  year_built integer null,  
  architect text null,  
  historical_significance text null,  
  fun_facts text[] null,  
  image_url text null,  
  website_url text null,  
  created_at timestamp with time zone not null default now(),  
  updated_at timestamp with time zone not null default now(),  
  constraint landmarks_pkey primary key (id)  
) TABLESPACE pg_default;  
  
create index IF not exists idx_landmarks_category  
on public.landmarks using btree (category) TABLESPACE pg_default;  
  
create index IF not exists idx_landmarks_location  
on public.landmarks using btree (latitude, longitude)  
TABLESPACE pg_default;
```

The schema is structured to support multiple functional layers:

- **Core identity and content:** unique ID, title, summary
- **Geographic rendering:** latitude, longitude
- **Categorization and interface logic:** category
- **Historical metadata:** year built, architect, historical significance, fun facts
- **Media and external resources:** image and website URLs
- **System management:** creation and update timestamps

Category and geographic indexes improve retrieval performance by supporting efficient category filtering and map-based landmark rendering.